



SY31 - Capteurs pour les systèmes intelligents Cartographie Guidée

Juin 2025

Binôme :

Evan Chevalérias
Étienne Lopez-Vilespy

Table des matières

0	Introduction	2
0.1	Contexte et objectifs du projet	2
0.2	Capteurs utilisés	2
0.3	Architecture du projet	3
1	Résultats et Analyse	4
1.1	Visualisations	4
1.1.1	Caméra	4
1.1.2	Trajectoire	5
1.1.3	Cartographie	6
2	Bilan et limites	8
3	Annexes	9
3.1	Architecture détaillées du projet	9
3.2	Répartition du travail	10

Chapitre 0

Introduction

0.1 Contexte et objectifs du projet

Ce projet, réalisé dans le cadre de l'UV SY31 (Capteurs pour les systèmes intelligents) à l'Université de Technologie de Compiègne, a pour but de réaliser une cartographie guidée d'un labyrinthe en utilisant un robot équipé d'un LiDAR. Le robot se déplace dans l'environnement et collecte des données de distance et de position, qui sont ensuite utilisées pour créer une carte, et donne des indications de navigation en fonction d'instructions dans l'environnement (des flèches pour guider le chemin). Un dépôt GitHub ([ici](#)) a été créé pour ce projet, où le code source et les instructions nécessaires à l'exécution sont disponibles.

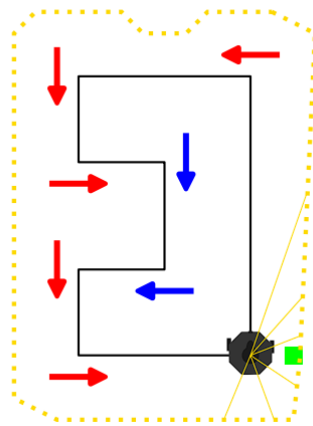


FIGURE 1 – Turtlebot naviguant dans un labyrinthe

Les objectifs de ce projet sont les suivants :

- Mettre en place un système de cartographie guidée à l'aide d'un robot mobile (Turtlebot3 Burger) équipé de capteurs (LiDAR, caméra, encodeurs, gyromètre).
- Permettre au robot de parcourir un environnement inconnu (labyrinthe) sous la supervision d'un opérateur humain.
- Accumuler les données de capteurs pour générer une carte de l'environnement en temps réel.
- Utiliser la caméra pour détecter des flèches de couleur indiquant la direction à suivre (gauche/droite) et assister l'opérateur dans la navigation.
- Analyser et visualiser les résultats obtenus afin d'évaluer la précision de la cartographie et l'efficacité de l'assistance à la navigation.

0.2 Capteurs utilisés

Les capteurs utilisés dans ce projet sont les suivants :

- **LiDAR (Light Detection and Ranging)** : utilisé pour mesurer les distances autour du robot et générer une carte de l'environnement sous forme de nuage de points.

- **Caméra** : permet de capturer des images de l’environnement, notamment pour détecter les flèches de navigation.
- **Encodeurs** : intégrés aux roues du robot, ils mesurent la rotation des roues pour estimer la distance parcourue et la position du robot (odométrie).
- **IMU (Inertial Measurement Unit) / Gyromètre** : fournit des informations sur l’orientation et la vitesse angulaire du robot, permettant d’améliorer l’estimation de la pose.
- **Sonar** : utilisé pour mesurer la distance aux obstacles proches, notamment pour déterminer la proximité d’un mur lors de la détection des flèches.

0.3 Architecture du projet

Le package du projet est composé de 5 nœuds ayant des objectifs bien précis :

- `decompressor.py` : Décompresse les données caméra du robot pour les rendre exploitables par `detect.py`.
- `detect.py` : Détecte les flèches de navigation dans les images issues de la caméra et publie la direction à suivre en conséquences, si le robot est proche du mur.
- `mapping.py` : Construit et met à jour la carte de l’environnement à partir des données du LiDAR et de la position du robot.
- `odom2pose.py` : Convertit les données d’odométrie (encodeurs et gyromètre) du robot en informations de pose (position et orientation), et trace la trajectoire du robot.
- `transformer.py` : Transforme les messages LiDAR du robot en nuages de points, utilisés pour la cartographie.

En Annexe se trouvent plus de détails quant à l’architecture du projet.

Chapitre 1

Résultats et Analyse

1.1 Visualisations

1.1.1 Caméra

La caméra est utilisée notamment pour détecter les flèches de navigation dans l'environnement. Les images capturées sont traitées pour identifier les flèches bleues et rouges, qui indiquent respectivement la direction à gauche et à droite. Un masque de couleur est appliqué pour isoler les flèches. La détection est effectuée en utilisant la bibliothèque OpenCV, qui permet de traiter les images en temps réel.

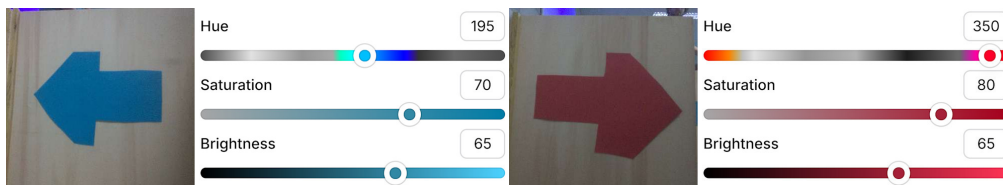


FIGURE 1.1 – Flèches dans le labyrinthe et couleurs

Pour mieux détecter les flèches rouges, on a combiné deux plages de couleur dans l'espace HSV. On a remarqué que cette double plage était nécessaire, car le rouge dans nos tests, selon la luminosité pouvait être situé aux deux extrémités du spectre de teinte. Ainsi, en combinant les deux masques, on a considérablement amélioré la fiabilité de la détection.

Ensuite, pour détecter le rouge et le bleu, on a utilisé la fonction `findContour` de la bibliothèque `cv2` qui nous permet de récupérer les éventuels contours des deux couleurs. Après cela, si les deux contours ne sont pas vides, on itère sur chaque contour en récupérant leur zone (area) avec la fonction `contourArea` de la même bibliothèque. Enfin, à chaque fois on vérifie quelle est la zone la plus grande de chaque couleur, après on peut comparer la plus grande zone "bleu" et "rouge", et on obtient la zone de couleur la plus prédominante.

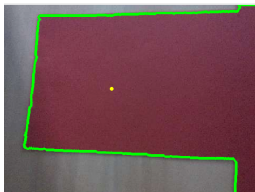


FIGURE 1.2 – Détection d'une flèche rouge

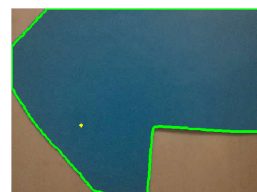


FIGURE 1.3 – Détection d'une flèche bleue

La détection d'une couleur va nous servir à indiquer la direction à prendre. Pour cela, nous affichons dans le terminal un message avec la direction adéquate. Pour être plus précis, à chaque détection de flèche, nous affichons une seule fois la direction quand on s'approche du mur, et une autre fois à partir d'un seuil qu'on considère critique, qui indique de tourner immédiatement. Via le sonar, qui renvoie le temps en μs que l'onde sonore a mis pour atteindre le mur et revenir, le calcul de la

distance au mur est effectué :

$$d = 340 \text{ m/s} \times \frac{dt \text{ s}}{2} \times 10^{-6}$$

où d est la distance au mur en mètres, et dt le temps mesuré par le sonar en microsecondes. Si la distance au mur est inférieure à un seuil critique (que nous avons fixé à 11 centimètres), le robot doit tourner immédiatement dans la direction indiquée par la flèche détectée. Sinon, il continue d'avancer tout droit.

```
[detect-5] [INFO] [1749482246.954568083] [detect]: Droite
[detect-5] [INFO] [1749482252.357491528] [detect]: Gauche
[detect-5] [INFO] [1749482252.410786096] [detect]: Gauche URGENT !
```

FIGURE 1.4 – Exemple d’affichage indiquant la direction

1.1.2 Trajectoire

En utilisant les données des encodeurs et du gyromètre on obtient les estimations des positions du robot suivantes (en bleu selon les encodeurs, en blanc selon le gyromètre) :

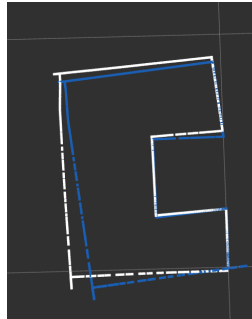


FIGURE 1.5 – Trajectoires (encodeurs et gyromètre)

Afin d’améliorer la précision de la trajectoire, les données des encodeurs et du gyromètre sont fusionnées en utilisant une moyenne pondérée, où les poids sont déterminés par les écarts-types des mesures. La formule de fusion pour x est la suivante (idem en y et en θ) :

$$x = \lambda x_{\text{enco}} + (1 - \lambda) x_{\text{gyro}}$$

$$\lambda = \frac{\sigma_{\text{gyro}}^2}{\sigma_{\text{enco}}^2 + \sigma_{\text{gyro}}^2}$$

Nous avons choisi les écarts-types suivants : $\sigma_{\text{enco}} = 0.2$ et $\sigma_{\text{gyro}} = 0.05$. Nous avons pris un écart-type plus grand pour les encodeurs car ils sont moins précis que le gyromètre, qui est plus sensible aux variations de position et d’orientation du robot.

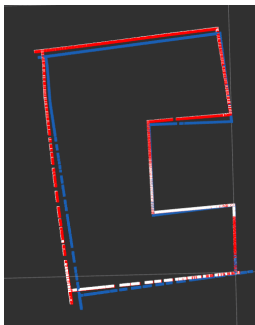


FIGURE 1.6 – Comparaison des trajectoires



FIGURE 1.7 – Moyenne pondérée

Dans la figure 1.6, on remarque bien la forme du labyrinthe et une meilleure approximation de la trajectoire du TurtleBot. Donc on en conclut que l’utilisation de la moyenne pondérée des données des encodeurs et du gyromètre permet d’améliorer significativement l’estimation de la position réelle du robot.

1.1.3 Cartographie

La cartographie est réalisée en utilisant les données du LiDAR et les informations de pose du robot (encodeurs et gyromètre). La première cartographie réalisée est considérée comme étant dans un repère fixe. Ainsi, les valeurs x_0 , y_0 et θ_0 sont fixées grâce aux valeurs de pose (encodeurs et gyromètre) à cet instant. Par la suite, les valeurs de x_t , y_t et θ_t sont calculées par rapport à cette origine via la transformation suivante :

$${}^0 \begin{bmatrix} x_1 & x_2 & \dots \\ y_1 & y_2 & \dots \\ 1 & 1 & \dots \end{bmatrix} = \begin{bmatrix} \cos \theta & -\sin \theta & x \\ \sin \theta & \cos \theta & y \\ 0 & 0 & 1 \end{bmatrix} \cdot {}^L \begin{bmatrix} x_1 & x_2 & \dots \\ y_1 & y_2 & \dots \\ 1 & 1 & \dots \end{bmatrix} \quad (1.1)$$

avec

- $x_1, x_2, \dots$ et $y_1, y_2, \dots$ les coordonnées des points du nuage de points relevé à l'instant t ,
- θ l'angle de rotation du robot entre le repère initial et le temps t , soit

$$\theta = \theta_t - \theta_0$$

- (x, y) la position du robot dans le repère fixe, soit

$$(x, y) = (x_t - x_0, y_t - y_0)$$

Comme pour la trajectoire, les données des encodeurs et du gyromètre sont fusionnées via une moyenne pondérée pour obtenir une pose plus précise, et donc une cartographie plus précise. La transformation est appliquée à chaque point du nuage de points LiDAR pour le placer dans le repère fixe.

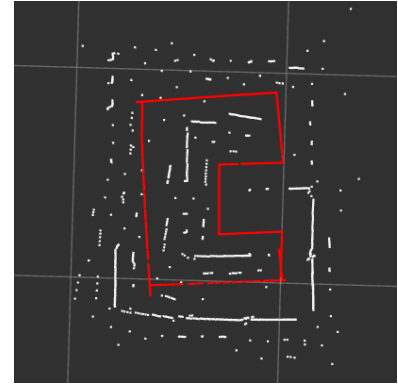
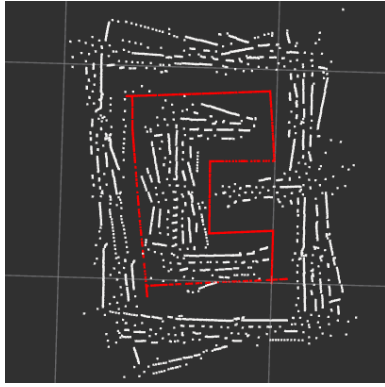
A chaque nouveau scan LiDAR, les points dont la distance avec leur plus proche voisin respectif est supérieure à un seuil prédéfini (que nous avons appelé `dist_min`, et que nous avons testé pour plusieurs valeurs allant de 0.03 à 0.1 mètre, voir ci-après) sont considérés comme des points n'ayant pas déjà été ajoutés précédemment, et sont donc ajoutés à la carte globale.

Comportement observé

Lors du développement de notre algorithme de cartographie, nous avons remarqué que le LiDAR du robot avait tendance à générer des points aberrants, se situant généralement dans les coins, et concernant les points très éloignés. Pour les supprimer, nous avons mis en place un filtre qui élimine les points dont la distance est supérieure à un seuil prédéfini (1 mètre), et dont les voisins consécutifs sont trop éloignés. Ce seuil a été choisi après plusieurs tests pour éviter d'éliminer des points valides tout en supprimant les points aberrants.

Cependant, nous avons aussi remarqué que le LiDAR avait tendance à se dérégler légèrement au fil du temps, entraînant des erreurs dans la cartographie. Pour compenser cela, nous avons tenté de mettre en place un algorithme de recalibrage qui ajuste la position du LiDAR en fonction des données précédemment collectées via un algorithme ICP (Iterative Closest Point) : cet algorithme permet de minimiser la distance entre les points du nuage de points actuel et ceux des nuages précédents, en ajustant la position du LiDAR.

L'objectif est d'appliquer une correction itérativement à chaque nouveau scan LiDAR, et la combiner avec la transformation de pose 1.1 pour obtenir une cartographie plus précise. Cependant, nous n'avons pas réussi à implémenter cet algorithme correctement dans le temps imparti, car les résultats obtenus étaient aléatoires et ne permettaient pas d'améliorer la précision de la cartographie avec certitude. Nous avons donc dû nous contenter de la cartographie sans recalibrage, causant une dégénérescence des points accumulés sur la carte au fil du temps. Voici ci-après un exemple de cartographie sans recalibrage, avec des seuils `dist_min` allant de 0.03 à 0.08 m.



Sans ICP : `dist_min` = 0.03 m

Sans ICP : `dist_min` = 0.05 m

Sans ICP : `dist_min` = 0.08 m

Ensuite, un exemple d'une cartographie au cours du temps avec recalibrage ICP, avec un seuil `dist_min` de 0.03 m :



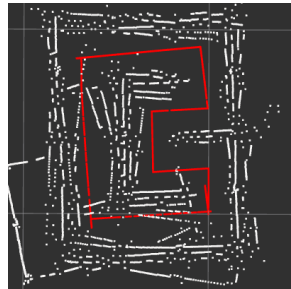
Avec ICP : 1/3 du par-
cours

Avec ICP : 1/2 du par-
cours

Avec ICP : 3/4 du par-
cours

Avec ICP : fin du par-
cours

Et enfin, plusieurs exemples de cartographies avec ICP, avec des seuils `dist_min` valant 0.03 m. On remarque que la cartographie est plus précise dans certains cas, mais un caractère aléatoire persiste.



Avec ICP : `dist_min` =
0.03 m

Avec ICP : `dist_min` =
0.03 m

Avec ICP : `dist_min` =
0.03 m

Avec ICP : `dist_min` =
0.03 m

Chapitre 2

Bilan et limites

Ce projet nous a permis de mettre en pratique les connaissances acquises dans le cadre de l’UV SY31, en développant un système de cartographie guidée à l’aide d’un robot mobile. Les objectifs initiaux ont été atteints, notamment la création d’une carte de l’environnement, un suivi de la trajectoire et la détection des flèches de navigation. Cependant, des améliorations sont possibles, notamment en ce qui concerne la précision de la cartographie.

L’intégration de l’algorithme ICP a mis en évidence un aspect aléatoire dans les résultats de la cartographie, rendant la précision variable d’une exécution à l’autre. Cette observation nous a permis de prendre conscience des limites pratiques de certains algorithmes de recalibrage, et de l’importance de bien maîtriser les paramètres et conditions d’application pour obtenir des résultats fiables.

Bien que nous aurions apprécié pouvoir implémenter un algorithme de recalibrage du LiDAR pour compenser les dérèglements observés, le travail fourni durant ce projet et les tentatives de mise en œuvre de cet algorithme nous ont permis d’approfondir notre compréhension des différents capteurs embarqués sur le robot (LiDAR, caméra, encodeurs, IMU, sonar), de leur fonctionnement, de leurs limites et de la manière dont leurs données peuvent être exploitées et fusionnées pour améliorer la perception et la navigation d’un système robotique. L’expérience acquise sur la gestion des données capteurs, leur synchronisation et leur traitement en temps réel constitue un apport précieux pour la suite de notre parcours.

Chapitre 3

Annexes

3.1 Architecture détaillées du projet

Le package du projet est composé de 5 nœuds ayant des objectifs bien précis. Ici sont détaillés les différents canaux d'abonnements et de publications de chaque nœud :

- **decompressor.py** : Décompresse les données caméra du robot pour les rendre exploitables par **detect.py**. Ce nœud est abonné au canal suivant :
 - **image_robot** (type **CompressedImage**) : reçoit les images compressées de la caméra du robot.Il publie sur les canaux suivants :
 - **image_raw** (type **Image**) : images décompressées de la caméra,
 - **camera_info** (type **CameraInfo**) : informations de la caméra, publiées à intervalles réguliers de 1 seconde.
- **detect.py** : Détecte les flèches de navigation dans les images issues de la caméra et publie la direction à suivre en conséquences, si le robot est proche du mur. Ce nœud est abonné aux canaux suivants :
 - **image_raw** (type **Image**) : reçoit les images décompressées de la caméra,
 - **sensor_state** (type **SensorState**) : obtient les mesures du sonar.Il publie sur les canaux suivants :
 - **detections** (type **Image**) : images de la caméra, annotées par les détections ou non,
 - **instructions** (type **String**) : instructions de navigation.
- **mapping.py** : Construit et met à jour la carte de l'environnement à partir des données du LiDAR et de la position du robot. Ce nœud est abonné aux canaux suivants :
 - **pose_enco** (type **PoseStamped**) : reçoit la pose calculée à partir des encodeurs dans le nœud **odom2pose.py**.
 - **pose_gyro** (type **PoseStamped**) : reçoit la pose calculée à partir du gyromètre dans le nœud **odom2pose.py**.
 - **points** (type **PointCloud2**) : reçoit les nuages de points transformés par le nœud **transformer.py**.Il publie sur les canaux suivants :
 - **map** (type **PointCloud2**) : carte globale générée à partir des données LiDAR et des poses des encodeurs et du gyromètre. Une méthode de fusion de données avec moyennes pondérées est utilisée pour combiner les poses des encodeurs et du gyromètre.
 - **map_enco** (type **PointCloud2**) : carte générée en utilisant uniquement la pose issue des encodeurs.
 - **map_gyro** (type **PointCloud2**) : carte générée en utilisant uniquement la pose issue du gyromètre.
- **odom2pose.py** : Convertit les données d'odométrie (encodeurs et gyromètre) du robot en informations de pose (position et orientation), et trace la trajectoire du robot. Ce nœud est abonné aux canaux suivants :
 - **imu** (type **Imu**) : reçoit les données du gyromètre.
 - **sensor_state** (type **SensorState**) : reçoit les données des encodeurs.Il publie sur les canaux suivants :
 - **pose_enco** (type **PoseStamped**) : position estimée à partir des encodeurs (sous forme de

- quaternions).
- `pose_gyro` (type `PoseStamped`) : position estimée à partir du gyromètre (sous forme de quaternions).
- `traj_enco` (type `PointCloud2`) : trajectoire estimée à partir des encodeurs.
- `traj_gyro` (type `PointCloud2`) : trajectoire estimée à partir du gyromètre.
- `traj_moy_enco_gyro` (type `PointCloud2`) : trajectoire estimée par fusion avec moyennes pondérées des deux sources.
- `transformer.py` : Transforme les messages LiDAR du robot en nuages de points, utilisés pour la cartographie. Ce nœud est abonné au canal suivant :
 - `scan` (type `LaserScan`) : reçoit les données brutes du LiDAR.Il publie sur le canal suivant :
 - `points` (type `PointCloud2`) : publie le nuage de points généré à partir des données du LiDAR.

3.2 Répartition du travail

- **Détection des flèches :**
 - Étienne Lopez–Vilesipy : 50%
 - Evan Chevalérias : 50%
- **Cartographie :**
 - Étienne Lopez–Vilesipy : 20%
 - Evan Chevalérias : 80%
- **Affichage et gestion des trajectoires :**
 - Étienne Lopez–Vilesipy : 100%
- **Rapport :**
 - Étienne Lopez–Vilesipy : 50%
 - Evan Chevalérias : 50%
- **Gestion du dépôt GitHub :**
 - Étienne Lopez–Vilesipy : 30%
 - Evan Chevalérias : 70%